

PORTADA

Análisis y Diseño de Sistemas II

Semana 10 — GRASP: Polymorphism, Pure Fabrication, Indirection y Protected Variations

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Hoy se cierra la lista completa
de los nueve patrones GRASP de Larman**

TRANSICIÓN

Cinco patrones ya vistos, cuatro por cerrar

El martes y el miércoles de esta semana vieron cinco de los nueve patrones GRASP (*General Responsibility Assignment Software Patterns*, de Craig Larman): **Information Expert**, **Creator**, **Controller**, **Low Coupling** y **High Cohesion**. No los repetimos hoy — se asumen ya entendidos.

Hoy cerramos los **cuatro patrones restantes**: **Polymorphism**, **Pure Fabrication**, **Indirection** y **Protected Variations**. Con estos cuatro se completa la introducción a GRASP y a la **Unidad 5 — Patrones de Diseño y Antipatrones**.

CONCEPTO · RETOMANDO LA SEMANA 7

De "polimorfismo" como mecanismo a Polymorphism como patrón

En la semana 7 (**Herencia, polimorfismo e interfaces**) vimos el **polimorfismo** como mecanismo de POO: distintas clases que comparten una interfaz común pueden responder de forma distinta al mismo mensaje, según su tipo concreto.

El patrón GRASP **Polymorphism** toma ese mecanismo y lo convierte en una **guía de asignación de responsabilidades**: cuando el comportamiento de una operación **varía según el tipo** de un objeto, esa responsabilidad debe asignarse usando **operaciones polimórficas** definidas en cada tipo — no mediante lógica condicional que pregunte por el tipo desde afuera.

CONTRAEJEMPLO

El síntoma: condicionales sobre el tipo

```
class CalculadorDeEnvio:
    def calcular(self, pedido):
        if pedido.tipo == "terrestre":
            return pedido.peso * 1200
        elif pedido.tipo == "aereo":
            return pedido.peso * 4500
        elif pedido.tipo == "maritimo":
            return pedido.peso * 800
        else:
            raise ValueError("tipo desconocido")
```

Cada vez que aparece un nuevo tipo de envío hay que **editar** este método y agregar una rama más. El conocimiento de "cómo se calcula cada tipo" vive centralizado en un único lugar que pregunta por el tipo, en vez de vivir en cada tipo.

CONCEPTO

La solución: cada tipo responde por sí mismo

```
abstract class Pedido:
    def calcularCosto(self): ...    # operación polimórfica

class PedidoTerrestre(Pedido):
    def calcularCosto(self):
        return self.peso * 1200

class PedidoAereo(Pedido):
    def calcularCosto(self):
        return self.peso * 4500

class PedidoMaritimo(Pedido):
    def calcularCosto(self):
        return self.peso * 800

# en cualquier punto del código:
total = pedido.calcularCosto()    # sin preguntar el tipo
```

Quien llama a `calcularCosto()` no necesita saber qué subtipo de `Pedido` tiene enfrente: cada subclase **sabe calcular su propio costo**. Agregar un nuevo tipo de envío significa **agregar una clase nueva**, no editar un método existente que ya funciona.

CONCEPTO · RELACIÓN CON LO VISTO

Polymorphism es el patrón detrás de OCP

En la semana 8 vimos **Open/Closed Principle (OCP)**: el diseño debe estar abierto a extensión, cerrado a modificación. El refactor anterior es exactamente eso en la práctica: `CalculadorDeEnvio` con el `if/elif` quedaba **cerrado a extensión** (había que modificarlo cada vez); la jerarquía de `Pedido` queda **abierta a extensión** (una clase nueva) y **cerrada a modificación** (las clases existentes no cambian).

Polymorphism es el patrón GRASP que explica *cómo* asignar responsabilidades para lograr ese efecto: repartiendo el comportamiento variable entre los tipos, en vez de centralizarlo en condicionales.

TRANSICIÓN

¿Y cuando ningún concepto del dominio encaja?

Polymorphism resuelve variación de comportamiento entre tipos que **ya existen** en el dominio. Pero hay responsabilidades — como guardar datos en un archivo, validar un formato o hacer un cálculo auxiliar complejo— que no le pertenecen naturalmente a ningún concepto del negocio. El segundo patrón de hoy resuelve justamente ese caso: **Pure Fabrication**.

CONCEPTO

Pure Fabrication: inventar una clase que no está en el dominio

Pure Fabrication ("fabricación pura") es el patrón que dice: cuando asignar una responsabilidad a una clase del dominio real violaría **High Cohesion** o **Low Coupling** (ya vistos esta semana), y ningún concepto del dominio encaja bien, se **inventa una clase artificial** que no representa nada del negocio, pero que organiza el diseño de forma limpia.

Ejemplos típicos: un `Repositorio` que persiste datos, un `Validador` que verifica reglas de formato, un `Calculador` que hace un cómputo auxiliar reutilizable. Ninguna de esas clases representa "algo" del mundo real que el negocio reconozca — son decisiones puramente de **diseño de software**.

CONCEPTO · EJEMPLO

Sin Pure Fabrication vs. con Pure Fabrication

```
// Sin Pure Fabrication: Estudiante mezcla su rol de dominio
// con la responsabilidad técnica de persistencia
class Estudiante:
    def guardarEnBaseDeDatos(self): ...    // baja cohesión, mucho acoplamiento

// Con Pure Fabrication: una clase inventada asume la persistencia
class RepositorioEstudiantes:             // no existe en el dominio real
    def guardar(self, estudiante): ...
    def buscarPorId(self, id): ...

class Estudiante:
    # solo conoce reglas de negocio de un estudiante
    ...
```

`RepositorioEstudiantes` no aparece en el vocabulario del negocio — nadie en la vida real habla de "el repositorio de estudiantes". Es una **clase de diseño**, no de dominio.

TENSIÓN CONCEPTUAL · RELACIÓN CON LO VISTO

Pure Fabrication tensiona el modelo de dominio

En la semana 3 (**Modelado conceptual**) insistimos en que el **modelo de dominio** solo debe contener conceptos que expertos del negocio reconocen — nada de detalles técnicos. `RepositorioEstudiantes` jamás debería aparecer en ese modelo conceptual.

La tensión es real y es sana: el **modelo de dominio** (análisis, semana 3) y el **modelo de diseño** (esta unidad) responden preguntas distintas. Pure Fabrication es una decisión legítima **en diseño** — nunca debería "colarse" hacia el modelo conceptual del negocio como si fuera un concepto real.

TRANSICIÓN

Desacoplar dos elementos que se necesitan pero no deberían conocerse

Pure Fabrication resuelve dónde poner una responsabilidad que no encaja en el dominio. El tercer patrón resuelve un problema distinto: dos elementos del sistema necesitan comunicarse, pero acoplarlos directamente los volvería frágiles entre sí. Ahí entra **Indirection**.

CONCEPTO

Indirection: un intermediario que desacopla

Indirection asigna la responsabilidad de mediar entre dos (o más) elementos a un **objeto intermediario**, para que esos elementos no se acoplen directamente entre sí. El intermediario típico es un **adaptador** o una **fachada**: una clase que traduce o simplifica el contacto entre el dominio y algo externo.

Ejemplo típico: el módulo de dominio no llama directamente a la librería de un proveedor de pagos externo (con su API particular); llama a un `AdaptadorDePagos` propio, y ese adaptador es el único que conoce los detalles de la librería externa.

CONCEPTO · RELACIÓN CON LO VISTO

Indirection normalmente se logra a través de una abstracción

```
interface PasarelaDePagos:
    def cobrar(self, monto): ...

class AdaptadorProveedorX implements PasarelaDePagos:
    def cobrar(self, monto):
        # traduce la llamada al SDK específico del proveedor X
        ...

class ServicioDeCompras:
    def __init__(self, pasarela: PasarelaDePagos):
        self.pasarela = pasarela          # depende del intermediario, no del proveedor
    def pagar(self, monto):
        self.pasarela.cobrar(monto)
```

Esto es exactamente lo que vimos en **DIP** la semana 9: `ServicioDeCompras` (alto nivel) no depende del SDK del proveedor (bajo nivel) — ambos dependen de `PasarelaDePagos`, la abstracción que actúa como intermediario. **Indirection es el patrón GRASP; DIP es el principio SOLID que explica por qué esa indirección debe apuntar hacia una abstracción y no hacia otro detalle concreto.**

TRANSICIÓN

El último patrón: el "paraguas" que los resume

Los tres patrones anteriores —Polymorphism, Pure Fabrication e Indirection— son, en el fondo, **técnicas concretas** para lograr algo más general: que el diseño resista el cambio. El cuarto patrón nombra esa idea general de forma explícita: **Protected Variations**.

CONCEPTO

Protected Variations: proteger el diseño de lo que va a cambiar

Protected Variations (PV) dice: identifiquen los **puntos de variación o inestabilidad previstos** en el diseño —lo que ya se sabe o se sospecha que va a cambiar— y encapsúlenlos detrás de una **interfaz estable**, de modo que el cambio quede contenido ahí y no se propague al resto del sistema.

Un **punto de variación** es cualquier parte del sistema donde se anticipa que la implementación cambiará con el tiempo: el proveedor de pagos, el motor de base de datos, el formato de un reporte, la regla de cálculo de un descuento.

CONCEPTO · SÍNTESIS DE LOS PATRONES ANTERIORES

PV es el "porqué"; los otros tres son el "cómo"

- **Polymorphism** protege el punto de variación "comportamiento según el tipo", usando jerarquías de clases.
- **Pure Fabrication** protege la cohesión del dominio creando una clase de diseño separada para lo que no encaja.
- **Indirection** protege contra el acoplamiento directo, insertando un intermediario.

Los tres son, en realidad, **formas específicas de aplicar Protected Variations**. Por eso Larman lo llama el patrón "paraguas": resume en una sola idea la motivación de **OCP** (semana 8, abierto a extensión) y de **DIP** (semana 9, depender de abstracciones) — proteger el diseño de cambios futuros.

INTERACCIÓN

¿Qué patrón GRASP aplica en cada caso?

En el chat, para cada situación identifiquen si el patrón más pertinente es **Polymorphism**, **Pure Fabrication**, **Indirection** o **Protected Variations**:

1. Un método calcula el descuento distinto según si el cliente es "regular", "frecuente" o "VIP", con un bloque `if/elif` larguísimo.
2. El equipo necesita registrar en un log cada operación importante, pero "Log" no es un concepto que el negocio reconozca.
3. El sistema hoy envía correos con una librería específica, y el negocio anticipa que el proveedor de correo puede cambiar en seis meses.
4. El módulo de reportes no debe conocer directamente el motor de base de datos que usa el módulo de persistencia.

5 minutos · respondan en el chat

El conjunto completo, de Larman

Patrón	Idea central
Information Expert	Asignar la responsabilidad a quien tiene la información necesaria
Creator	Quién debe crear una instancia, según quién la contiene o usa
Controller	Un objeto no-UI recibe y coordina eventos del sistema
Low Coupling	Minimizar dependencias entre elementos
High Cohesion	Cada clase con responsabilidades enfocadas y relacionadas
Polymorphism	Comportamiento variable por tipo, resuelto con operaciones polimórficas
Pure Fabrication	Clase inventada, sin equivalente en el dominio, para mantener cohesión/acoplamiento
Indirection	Un intermediario desacopla dos elementos
Protected Variations	Encapsular puntos de variación previstos detrás de una interfaz estable

SÍNTESIS · CIERRE DE LA INTRODUCCIÓN A GRASP

Los cuatro patrones de hoy, en una frase

1. **Polymorphism:** reemplaza condicionales sobre el tipo por operaciones polimórficas — es el patrón detrás de OCP.
2. **Pure Fabrication:** inventa una clase de diseño (Repositorio, Validador, Calculador) cuando ningún concepto del dominio encaja sin dañar cohesión o acoplamiento.
3. **Indirection:** un objeto intermediario (adaptador, fachada) desacopla dos elementos — se logra normalmente a través de una abstracción, como en DIP.
4. **Protected Variations:** el patrón "paraguas" — encapsular puntos de variación previstos detrás de una interfaz estable, resumiendo la motivación de OCP y DIP.



CIERRE

Con esto se cierra la introducción a GRASP

En dos sesiones vieron los **nueve patrones GRASP** completos: cinco el martes y miércoles (Information Expert, Creator, Controller, Low Coupling, High Cohesion) y cuatro hoy (Polymorphism, Pure Fabrication, Indirection, Protected Variations).

La **semana 11** se usa este vocabulario de patrones para modelar en **UML dinámico y estático** el diseño de detalle del sistema. El **viernes de la semana 11** hay un **laboratorio de aplicación de GRASP** directamente sobre el proyecto propio de cada equipo.

